College AI Assistant: A Multi-Signal, Domain-Constrained Conversational System

Joseph Jonathan Fernandes

Applications of AI Project Report

***Abstract*—This report presents a college-domain conversational assistant that combines local multi-signal NLU control (preprocessing, intent, scope, emotion, and context) with external LLM response generation. Component-level evaluation on repository intent patterns shows strong semantic-routing performance (accuracy 0.783, macro F1 0.756), while end-to-end evaluation on 120 scenarios captures deployment-facing behavior under noisy, vague, emotional, and out-of-scope (OOS) inputs. A key finding is that a substantial portion of observed failures in batch execution are infrastructure-related (API rate-limit/provider failures) rather than core routing logic errors. Separating infrastructure failures from functional failures yields a clearer estimate of system behavior and supports more accurate interpretation of confidence intervals, OOS gating quality, and latency trade-offs. The primary contribution is a deployment-realistic evaluation methodology that separates infrastructure failures from functional routing errors, yielding corrected confidence intervals that better reflect core assistant logic.**

I. Literature Survey

Conversational AI systems are commonly organized as (i) an NLU layer that maps user language to structured intent/state and (ii) a response layer that produces natural language outputs. The literature has progressively shifted from model-specific improvements toward system-level designs that balance controllability, coverage, and deployment realism. A consistent taxonomy is therefore foundational: Hussain et al. (2019) categorize agents into rule-based, retrieval-based, and generative families [1], while Caldarini et al. (2022) document the rapid diversification of architectures and the corresponding fragmentation of evaluation practices in recent years [2]. Gao et al. (2019) reinforce that conversational AI performance is determined by the interaction among NLU, state tracking, response generation, and evaluation, motivating integrated pipelines rather than purely model-centric narratives [3].

A. Traditional Methods → Transformers → Dialogue Systems → Intent Evaluation

Early neural NLU work targets a core limitation of modular pipelines: error propagation between intent classification and slot filling. Liu and Lane (2016) propose an attention-based bidirectional RNN trained jointly for utterance-level intent and token-level slot tagging [4], and Hakkani-Tur et al. (2016) extend joint modeling to multi-domain semantic frame parsing with BiLSTM-style encoders to support transfer across domains [5]. These models provide stronger representations than hand-engineered features but remain constrained by sequential recurrence (limiting parallelism and long-context modeling) and by fixed ontologies and domain boundaries, which become brittle as domain coverage expands.

Transformers address key sequence-modeling bottlenecks by replacing recurrence with self-attention. Vaswani et al. (2017) introduce the encoder–decoder transformer with multi-head scaled dot-product attention and positional encodings, enabling parallel training and stronger long-range interactions but with quadratic $O(n^2)$ attention cost in sequence length [6]. On this foundation, Devlin et al. (2019) show that bidirectional masked language model pre-training (paired with next sentence prediction) yields transfer-ready contextual representations that substantially improve downstream classification and extraction tasks [7]. Liu et al. (2019) further demonstrate that many gains arise from pre-training recipes: removing next sentence prediction, training longer on more data with larger batches, and using dynamic masking produces consistent improvements in encoder representations [8]. However, stronger encoders alone do not remove deployment needs for domain scoping, OOS detection, and uncertainty-aware decision thresholds.

Advances in response generation follow a complementary trajectory. Autoregressive transformer decoders (Radford et al., 2018, 2019) establish generative pre-training as a practical foundation for response-layer modeling, enabling adaptation by fine-tuning or prompt conditioning [9], [10]. Brown et al. (2020) show that scaling enables in-context few-shot learning, offering a prompt-based adaptation interface without gradient updates, but with high inference cost and limited grounding that can yield fluent but incorrect outputs [11]. Alignment and dialogue specialization subsequently target usability and safety: Ouyang et al. (2022) describe RLHF via supervised fine-tuning, reward modeling from human preferences, and policy optimization [12]; Thoppilan et al. (2022) demonstrate dialogue-optimized training objectives emphasizing qualities such as safety and groundedness [13]; and OpenAI (2022) illustrates the practical impact of dialogue-aligned assistants while noting that alignment does not guarantee factual grounding, particularly in restricted domains [14].

System-level deployments and empirical "recipes" emphasize that these generative capabilities must be embedded within orchestration and evaluation discipline. The Alexa Prize reveals practical challenges in sustained open-domain dialogue and motivates hybrid architectures that combine retrieval, generation, topic tracking, and multi-skill routing,

while also highlighting that real-user ratings are noisy and context-dependent [15]. Roller et al. (2021) similarly show that blended retrieval–generation systems can outperform single-strategy approaches across human-evaluated quality dimensions, though many techniques assume substantial data and annotation resources [16].

B. Intent and Dialogue Understanding Components

Intent and dialogue understanding remain central because assistant behavior depends on reliably mapping user utterances to structured decisions. Transformer-based joint NLU approaches such as Chen et al. (2019) fine-tune BERT to jointly perform intent classification (via [CLS]) and slot labeling (token-level tagging, often paired with a CRF), yielding strong baselines on standard benchmarks [17]. Alternative structured inductive biases, such as capsule networks with dynamic routing, explicitly model hierarchical relationships between tokens, slots, and intents [18], but can add computational complexity relative to standard encoders.

Deployment constraints motivate efficiency-focused intent modeling. Casanueva et al. (2020) demonstrate that dual sentence encoders with similarity-based classification provide an efficient intent detection alternative in the Banking77 setting [19]. Meanwhile, continual domain growth and limited supervision motivate few-shot approaches: Zhang et al. (2020) use contrastive pre-training to learn intent-discriminative representations, followed by few-shot fine-tuning to improve generalization when only a handful of labeled examples exist [20]. Benchmark design further shapes conclusions about robustness; CLINC150 explicitly incorporates OOS inputs, requiring models to reject unsupported queries and motivating calibrated confidence thresholds for deployment [21].

Multi-turn dialogue introduces state-tracking requirements beyond single-utterance intent classification. Kim et al. (2019) reframe dialogue state tracking as reading comprehension over dialogue history, enabling extraction of open-vocabulary values that may not appear in training [22]. Huang et al. (2020) propose TRADE, a transferable dialogue state generator with a copy mechanism that generates slot values token-by-token from context, improving cross-domain transfer but introducing latency and potential degradation on long contexts [23].

Evaluation practices motivate multi-dimensional reporting rather than reliance on a single score. Maroengsit et al. (2019) survey automatic metrics and human evaluation protocols, emphasizing that many automatic metrics correlate weakly with human judgments in open-domain dialogue [24].

C. Dataset Artifacts, Robustness, and Deployment Risk

Several lines of work show that dataset properties can inflate apparent progress and complicate generalization claims. Gururangan et al. (2018) identify annotation artifacts that allow models to exploit shallow correlations rather than the intended semantics [25], and Geva et al. (2019) demonstrate shortcut learning arising from annotator-specific patterns [26]. Swayamdipta et al. (2020) further show that aggregate accuracy obscures systematic difficulty regimes by mapping instances using training dynamics, enabling more diagnostic evaluation of easy, ambiguous, and hard examples [27].

Robustness under realistic noise is also central for deployed assistants. Belinkov and Bisk (2018) quantify sharp degradation in neural models under character-level perturbations such as typos [28], while Sun et al. (2020) show that adversarial training can improve resilience for multilingual and code-mixed dialogue inputs, albeit with added data and compute costs [29]. These findings motivate conservative preprocessing and explicit control policies alongside statistical models.

Collectively, the surveyed literature motivates a hybrid architecture in which deterministic control (intent routing, scope gating, and monitoring) complements generative response modeling. The following section formalizes this design for a college-domain assistant and specifies the pipeline and decision points that guide the repository implementation.

II. Methodology (Design)

The system is designed as a domain-constrained conversational assistant that combines a structured NLU/control layer with a controlled response generation layer. This hybrid approach follows system-level findings that robust assistants require orchestration, context handling, uncertainty management, and evaluation discipline in addition to fluent text generation [2], [3], [15], [24], [30].

A. System Overview

The design targets a college-information domain and is organized as a multi-signal pipeline:

- **Control and understanding:** preprocessing, intent inference, uncertainty estimation, scope (in-scope/OOS) decisions, and conversation context.
- **Response layer:** knowledge-grounded response planning followed by controlled natural language generation.
- **Monitoring and evaluation:** telemetry signals (intent confidence, latency, error modes) and test-driven regression checks to track reliability.

This decomposition reflects common taxonomies for conversational agents and supports predictable behavior under ambiguous or out-of-scope inputs [1], [19], [21].

B. Architecture Diagram (Figure 1)

Figure 1 summarizes the repository-grounded architecture. The design separates local control decisions (preprocessing, intent/emotion/scope inference, context) from external LLM response generation. This separation aligns with deployment guidance emphasizing modular guardrails and explicit control points for conversational systems [12], [15], [16].

C. Data Flow Pipeline

The end-to-end pipeline is defined as a deterministic sequence of transformations and decisions:

Input $\rightarrow$ **Preprocessing** $\rightarrow$ **Representation** $\rightarrow$ **Intent/Scope Decision** $\rightarrow$ **Response Planning** $\rightarrow$ **Generation** $\rightarrow$ **Output**

More explicitly:

TABLE I: Comparison Table (Selected Works)

Paper	Method	Strength	Limitation
[6]	Multi-head self-attention encoder-decoder	Parallel training and long-range dependency modeling	$O(n^2)$ attention cost
[4]	BiRNN joint intent+slot architecture	Early strong baseline for integrated NLU	Sequential recurrence limits throughput
[7]	110M-parameter bidirectional encoder, masked-LM pre-training, fine-tuned [CLS] for intent	Strong transferable contextual representations	Encoder-only; needs calibration for OOS
[17]	BERT + token-level slot tagging (joint NLU)	Strong intent/slot benchmark performance	Heavier inference than lightweight baselines
[20]	Contrastive pre-training + few-shot intent fine-tuning	Better low-resource intent onboarding	Sensitive to support-set quality
[19]	Dual sentence encoders with cosine intent retrieval	Fast intent detection with efficient embedding lookup	Domain-specific evaluation scope
[21]	CLINC150 in-scope/OOS benchmark protocol	Explicitly evaluates rejection behavior	OOS definition varies by deployment
[22]	Dialogue state tracking as reading comprehension	Open-vocabulary slot value extraction	Higher per-turn computational cost
[23]	TRADE dialogue state generator with copy mechanism	Strong cross-domain slot-value transfer with open vocabulary	Added latency and degradation risk on long dialogue contexts
[10]	Autoregressive decoder pre-training	Fluent open-ended generation	Hallucination/grounding risk
[16]	Retrieval-generation hybrid chatbot recipes	Strong practical recipe for open-domain systems	Data and infrastructure intensive
[12]	SFT + reward model + PPO alignment pipeline	Better instruction following and safety behavior	Expensive and preference-bias sensitive
[13]	Dialogue-specialized LM training objectives	Improved conversational quality dimensions	Resource-intensive; residual factual errors
[25]	Annotation-artifact analysis in NLU datasets	Exposes spurious dataset shortcuts	Not a direct mitigation method
[26]	Annotator-bias shortcut learning analysis	Diagnoses hidden labeling bias effects	Requires downstream debiasing work
[27]	Dataset cartography via training dynamics	Identifies easy/ambiguous/hard instances	Adds analysis overhead
[28]	Character-noise robustness evaluation	Quantifies brittleness under perturbations	Improvement methods required separately
[29]	Adversarial training for multilingual/code-mixed dialogue robustness	Improves resilience to noisy code-mixed perturbations	Requires additional adversarial data and training compute

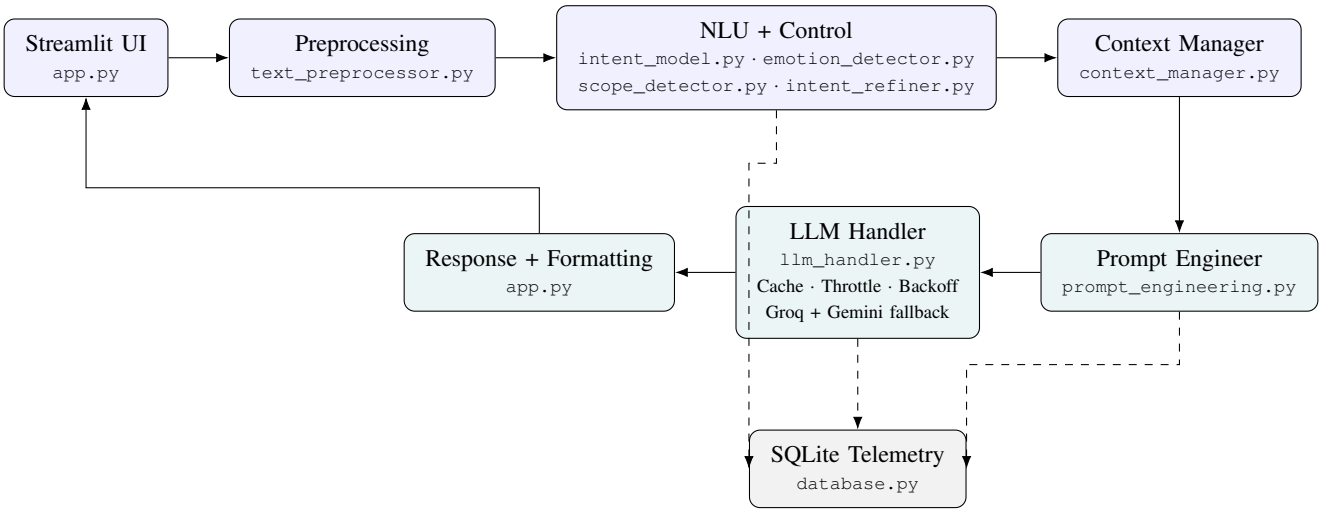

Fig. 1: Architecture of the college-domain assistant. Blue nodes run locally (NLU/control layer); teal nodes delegate to the external LLM endpoint (generation layer). Dashed lines indicate telemetry writes to SQLite.

1) **Input:** user utterance u_t.
2) **Preprocessing:** normalization to reduce sensitivity to character-level noise that can degrade NLU performance [28], [29].
3) **Representation:** a lexical vector $v(u_t)$ (sparse) and a semantic embedding $e(u_t)$ (transformer encoder) [7], [8].
4) **Intent inference:** compute intent scores and aggregate them into a calibrated posterior $p(y \mid u_t)$ over intents.
5) **Uncertainty and scope:** compute confidence $c = \max_y p(y \mid u_t)$ and apply thresholds to accept, clarify, or reject (OOS) [19], [21].
6) **Response planning:** condition on (u_t, m_t) and retrieved facts k_t to form a response plan that constrains generation.
7) **Generation and output:** produce the final natural language response with conservative behavior under uncertainty [10], [12], [13].

D. Model Design (intent classification + response generation)

1) Intent classification: Intent classification is modeled as multi-class prediction over a closed intent set. Given tokenized input $x = [x_1, \ldots, x_n]$, a transformer encoder produces contextual states $H = [h_1, \ldots, h_n]$ [6], [7]. A pooled vector z is mapped to a posterior over intents:

$$p(y \mid x) = \mathrm{softmax}(Wz + b), \quad c = \max_y p(y \mid x).$$

For efficiency, inference may be complemented with similarity-based retrieval in embedding space, where each intent i has a prototype text t_i and score

$$s_i = \cos\big(e(u_t), e(t_i)\big), \quad \hat{y} = \arg\max_i s_i.$$

The final decision uses uncertainty-aware thresholds (e.g., reject if $c < \tau$ or $\max_i s_i < \gamma$), reflecting the need for calibrated OOS behavior [19]–[21]. In the implemented system, τ is intent-specific via dynamic confidence thresholds (typical academic intents in the 0.45–0.60 range from `confidence_threshold_manager.py`), while the semantic scope gate uses $\gamma = 0.50$ (from `scope_detector.py`); this setting directly affects observed OOS precision/recall.

2) Response generation: Response generation is treated as conditional text generation using an autoregressive decoder-style language model (LM). Given a structured response plan and grounding context, generation models the probability of an output sequence $r = [r_1, \ldots, r_T]$ as

$$p(r \mid u_t, m_t, k_t) = \prod_{t=1}^{T} p(r_t \mid r_{<t}, u_t, m_t, k_t).$$

Decoder-only pre-training establishes the base generative capability [9]–[11]. Alignment and dialogue specialization approaches improve instruction adherence and conversational quality [12]–[14], but they do not guarantee domain factuality; therefore, grounding and scope control remain first-class design requirements [15], [16].

E. Algorithms / Models Used (RNN vs Transformer comparison)

Traditional joint NLU systems often rely on recurrent encoders (BiRNN/BiLSTM) trained jointly for intent and slots [4], [5]. Recurrence provides a strong inductive bias for local sequential patterns but is inherently sequential and can limit throughput.

Transformer encoders replace recurrence with self-attention, enabling parallel training and stronger long-range context modeling [6]. Pre-trained transformer encoders (e.g., BERT/RoBERTa) provide transferable representations that improve intent classification and joint intent/slot baselines [7], [8], [17].

For response generation, decoder-style transformers support fluent multi-turn responses but require alignment and system constraints to mitigate hallucination risk [10], [12]. End-to-end trainable dialogue systems have also been explored, including task-oriented dialogue networks and memory-based approaches [31], [32]; however, these formulations provide fewer explicit control points for domain scoping and safety policies.

F. Design Justification (why chosen)

The design adopts a modular, hybrid architecture rather than a fully end-to-end conversational model because modular decomposition improves controllability and allows explicit enforcement of domain scope, uncertainty handling, and evaluation checkpoints [1], [3], [15], [16].

Uncertainty-aware intent routing is emphasized because deployed assistants must handle ambiguous and out-of-scope inputs; explicit OOS gating is supported by intent benchmarks designed to measure rejection performance [19], [21]. A hybrid intent stack (lexical evidence + transformer-derived semantics) supports a practical trade-off among interpretability, latency, and paraphrase robustness [7], [8].

Robustness and measurement risks motivate conservative preprocessing and careful interpretation of automated evaluation results. Neural NLU brittleness under noise and dataset artifacts can inflate reported performance without improving real-world reliability [25]–[29]. The methodology therefore prioritizes scope control and grounded generation over unconstrained fluency, consistent with survey evidence that reliable chatbot performance depends on system integration and evaluation discipline [2], [24]. The following section maps this design to concrete modules and execution pathways in the repository.

III. Implementation

This section operationalizes the design using Python modules in the repository. The implementation emphasizes reproducible preprocessing, explicit scope and confidence controls, and a monitored LLM response layer.

A. Development Environment (languages, tools, libraries)

Language and runtime. The system is implemented in Python and executed as either a Streamlit web application (interactive UI) or a command-line runner.

Core libraries.

- **UI:** Streamlit for the chat interface and session state management.
- **NLU models:** PyTorch as the deep learning backend, Hugging Face Transformers for transformer pipelines, and Sentence-Transformers for embedding-based semantic similarity.
- **Classical ML:** scikit-learn for TF–IDF vectorization and logistic regression.
- **Data and utilities:** NumPy/Pandas for numeric and tabular handling; Requests for HTTP.
- **LLM providers:** Groq client/HTTP for the primary model endpoint and `google-genai` for Gemini fallback; `python-dotenv` for environment-variable configuration.
- **Persistence:** SQLite (via the Python standard library) for interaction logging and lightweight analytics.

Testing and quality tools. The development toolchain includes PyTest (with coverage, parallelism, and timeouts), Hypothesis for property-based tests, Locust for load/performance testing, and formatting/linting tools (Black, Flake8, isort).

TABLE II: Illustrative Hinglish/code-switch normalization examples.

Raw query	Normalized query
fees kitni hai	what are the fees
kya placement hoti hai	what placement is
hostel me room hai kya	hostel room is what
exam kab hai	exam when is
scholarship kaise milegi	scholarship how get

B. Dataset Description (source, preprocessing steps)

Two local JSON artifacts serve as the primary data sources:

- **Intent dataset** (`data/intents.json`): an intent schema containing tags, paraphrased training patterns, and optional keyword lists. This file is used to train lightweight intent components at application startup.
- **Domain knowledge base** (`data/college_data.json`): structured college information (fees, exams, placements, hostel, admissions, etc.) used for retrieval of relevant facts for response grounding.

Preprocessing. User input and intent patterns are normalized using a dedicated preprocessing stage (`text_preprocessor.py`). Key transformations include typo normalization and abbreviation expansion, Hinglish/code-switch mapping for common Hindi–English tokens, and regex-based cleanup for punctuation artifacts and whitespace normalization. This preprocessing targets practical robustness issues where minor orthographic noise can degrade NLU signals [28], [29].

Normalization preserves content words and discards many Hindi function words; downstream NLU relies on semantic embeddings rather than grammaticality.

C. Model Implementation (training, fine-tuning)

The implementation uses pre-trained transformer models without project-specific fine-tuning of large encoders/decoders. Instead, fast-to-train components are fit on local intent patterns, while response generation relies on instruction-following LLM inference.

1) Intent components (startup training):

- **Semantic classifier:** a Sentence-Transformers encoder (default: `all-MiniLM-L6-v2`) embeds each preprocessed intent pattern; inference selects the intent with maximal cosine similarity to the user embedding.
- **TF–IDF classifier:** a scikit-learn TF–IDF vectorizer (unigrams/bigrams) and logistic regression classifier are trained on the same preprocessed patterns.
- **Ensembling and calibration:** the final intent prediction uses weighted aggregation with explicit calibration heuristics (agreement boosts, scaling factors, confidence floors) to improve stability under paraphrases and short-/noisy inputs.

This hybrid intent implementation reflects the practical combination of transformer representations [6]–[8] and efficient similarity-based intent matching [19].

2) Emotion signal (pre-trained inference): A cached transformer pipeline performs sentiment analysis and maps sentiment to coarse interaction emotions (e.g., neutral, stressed). Keyword-based filters reduce false positives on short information-seeking questions.

3) LLM response generation (inference, no fine-tuning): Response generation is performed via an autoregressive LLM endpoint (Groq, with Gemini fallback), consistent with decoder-only pre-training and instruction-following alignment approaches [10], [12], [14]. The implementation does not train a dialogue model from scratch; it conditions inference using structured prompts and retrieved domain facts.

D. System Integration (frontend/backend if any)

Frontend. The Streamlit application orchestrates the end-to-end turn loop (message capture, inference calls, UI rendering) and maintains session identifiers for logging.

Backend orchestration. The orchestration layer instantiates and caches major components (intent, scope, emotion, prompt builder, LLM handler, and context manager) to amortize model-loading cost.

Persistence and analytics. A SQLite-backed logging layer records timestamped interactions with intent/confidence, emotion, scope decisions, response time, and LLM source. A small connection pool reduces per-turn database overhead.

E. Key Functional Modules

1) Intent classification: Intent classification is implemented as an ensemble: semantic similarity over transformer sentence embeddings; TF–IDF + logistic regression for lexical pattern capture; and a context-aware intent refiner that adjusts ambiguous predictions using recent-turn intent continuity.

Pipeline (pseudo-code style).

1) $u_t \leftarrow$ user_message
2) $u'_t \leftarrow \text{preprocess}(u_t)$
3) $(y_{\text{sem}}, c_{\text{sem}}) \leftarrow \text{semantic_predict}(u'_t)$
4) $(y_{\text{lex}}, c_{\text{lex}}) \leftarrow \text{tfidf_predict}(u'_t)$
5) $(y, c) \leftarrow \text{weighted_ensemble}(y_{\text{sem}}, c_{\text{sem}}, y_{\text{lex}}, c_{\text{lex}})$
6) $(y, c) \leftarrow \text{refine_with_history}(y, c, \text{history}, \text{emotion})$

To support deployment realism, the intent decision is followed by explicit confidence thresholding and OOS handling aligned with benchmarks that emphasize rejection performance [21].

2) Entity/slot extraction: The implementation includes lightweight entity extraction in the context manager to support multi-turn continuity rather than full named-entity recognition. Extracted entities include user name patterns and intent-specific fields (e.g., department/program hints, exam type, fee-related subtopics). Pronoun resolution uses the most recent tracked entities to reduce context loss in follow-up turns.

3) Response generation: Response generation is implemented as a controlled LLM call:

- **Prompt construction:** a prompt-engineering module injects detected intent, confidence, scope status, emotion, conversation history, and retrieved knowledge snippets.

- **Knowledge grounding:** relevant sections of the college JSON are selected as context, with fallback to generic templates when a requested field is absent.
- **Operational safeguards:** caching for repeated queries, throttling between calls, concurrency limiting, exponential backoff with jitter, and API-key rotation; if the primary provider fails, the system falls back to Gemini.

These controls are implemented to reduce hallucination risk and stabilize user experience under rate limits and variable latency [15], [16].

F. Selected Core Code Blocks

The following snippets omit imports, UI code, error-handling boilerplate, and comments.

Data Normalization

```
@staticmethod
def remove_special_chars(text: str) -> str:
        text = re.sub(r"[^a-z0-9\s\?!]", "", text)
        text = re.sub(r"\s+", " ", text).strip()
        return text

@staticmethod
def normalize_hinglish(text: str) -> str:
        words = text.split()
        converted = []
        for word in words:
                clean_word = word.rstrip("?!")
                ending = word[len(clean_word):]
                if clean_word in
     TextPreprocessor.HINGLISH_MAP:
                        mapped =
     TextPreprocessor.HINGLISH_MAP[clean_word]
                        if mapped:
                                converted.append(mapped +
     ending)
                else:
                        converted.append(word)
        return " ".join(converted)
```

Hybrid Intent Ensemble

```
sem_weighted = sem_conf * self.semantic_weight
tfidf_weighted = tfidf_conf * self.tfidf_weight

if sem_intent == tfidf_intent:
        final_intent = sem_intent
        avg_conf = (sem_conf + tfidf_conf) / 2.0
        agreement_boost = 1.15
        final_confidence = min(1.0, avg_conf *
     agreement_boost * self.confidence_scaling)
elif max(sem_weighted, tfidf_weighted) > 0.6:
        if sem_weighted > tfidf_weighted:
                final_intent = sem_intent
                final_confidence = min(1.0, sem_weighted *
     self.confidence_scaling)
        else:
                final_intent = tfidf_intent
                final_confidence = min(1.0, tfidf_weighted
     * self.confidence_scaling)
else:
        if sem_weighted >= tfidf_weighted:
                final_intent = sem_intent
                final_confidence = min(1.0, sem_weighted *
     self.confidence_scaling)
        else:
                final_intent = tfidf_intent
                final_confidence = min(1.0, tfidf_weighted
     * self.confidence_scaling)

final_confidence = max(self.confidence_min,
     final_confidence)
return final_intent or "unknown", final_confidence,
     detailed_scores
```

Uncertainty and Scope Gating

```
query_lower = query.lower()
has_college_context = any(prefix in query_lower for prefix
     in self.COLLEGE_CONTEXT_PREFIXES)
if has_college_context:
        return True, "college_context_detected", 0.85

if self.definite_oos_pattern.search(query_lower):
        return False, "definite_out_of_domain", 0.95
if self.ambiguous_oos_pattern.search(query_lower):
        return False, "ambiguous_out_of_domain", 0.65

domain_score, matched_category =
     self._check_domain_keywords(query_lower)
if self.use_semantic and domain_score == 0.0:
        semantic_score, semantic_category =
     self._check_semantic_similarity(query)
        if semantic_score > domain_score:
                domain_score, matched_category =
     semantic_score, semantic_category

if conversation_history:
        context_boost =
     self._compute_context_score(query_lower,
     conversation_history)
        if context_boost > 0:
                domain_score = min(0.95, domain_score +
     context_boost)

if domain_score > self.confidence_threshold:
        return True, f"domain_keywords_{matched_category}",
     domain_score
return False, "low_domain_confidence", max(domain_score,
     intent_confidence * 0.3)
```

Context Tracking

```
def add_turn(self, user_input, bot_response, intent,
     confidence, emotion, entities=None):
        turn = {
                "timestamp": datetime.now().isoformat(),
                "user_input": user_input,
                "bot_response": bot_response,
                "intent": intent,
                "confidence": confidence,
                "emotion": emotion,
                "entities": entities or {}
        }
        self.conversation_history.append(turn)
        self.last_intent = intent
        self.last_confidence = confidence
        self.last_emotion = emotion
        self.last_entities = entities or {}

def resolve_pronouns(self, user_input: str) -> str:
        lower_input = user_input.lower()
        resolved = user_input
        if ("it" in lower_input or "that" in lower_input or
     "this" in lower_input) and self.last_entities:
                if "department" in self.last_entities:
                        resolved = resolved.replace("it",
     self.last_entities["department"])
                        resolved = resolved.replace("that",
     self.last_entities["department"])
        return resolved
```

LLM Operational Safeguards

```
last_error = None
for attempt in range(self.retry_max_attempts):
        try:
                response = requests.post(

     f"{self.groq_base_url}/chat/completions",
                        headers=headers,
                        json=payload,
                        timeout=self.groq_timeout
                )

                if response.status_code == 429:
                        self.rate_limit_count += 1
```

```
                    if len(self.groq_api_keys) > 1:
                            self._rotate_groq_key()
                            groq_api_key =
self._get_current_groq_key()

time.sleep(self._get_jittered_backoff_delay(attempt))
                            continue
                    if attempt <
self.retry_max_attempts - 1:

time.sleep(self._get_jittered_backoff_delay(attempt))
                            continue
                    return {"error": "Rate limit
exceeded after retries"}

            if response.status_code != 200:
                    if 500 <= response.status_code <
600 and attempt < self.retry_max_attempts - 1:

time.sleep(self._get_jittered_backoff_delay(attempt))
                            continue
                    return {"error": last_error}
```

G. Challenges Faced and Solutions

Noisy and code-mixed inputs. Short, misspelled, or Hinglish queries can degrade lexical and embedding signals. A dedicated normalization layer and dual-model ensemble mitigate brittleness [28], [29].

Out-of-scope (OOS) behavior. Closed-set intent classifiers are prone to overconfident misrouting for unsupported requests. The implementation includes explicit scope detection and confidence-based gating, consistent with OOS-oriented intent benchmarks [19], [21].

Latency and rate limiting for LLM calls. External LLM endpoints introduce variable response times and occasional rate limits. The implementation uses caching, throttling, concurrency limits, and retry/backoff policies, with provider fallback to maintain responsiveness.

Multi-turn coherence. Follow-up questions often omit explicit context (e.g., "What about that?"). A bounded conversation history and lightweight entity tracking reduce context loss without requiring full dialogue-state tracking models.

This implementation provides the concrete basis for the empirical results reported in the following section.

IV. Results

This section reports quantitative outcomes from (i) a component-level intent-classification evaluation using the repository intent patterns and (ii) an end-to-end automated test run recorded in a saved execution artifact. Metrics and interpretations follow established guidance that chatbot evaluation should separate component accuracy from overall conversational quality and should account for dataset artifacts and measurement bias [24]–[27].

A. Evaluation Metrics (Accuracy, Precision, Recall, F1-score)

Two metric groups are reported:

- **Intent classification (component-level):** Accuracy and macro-averaged Precision/Recall/F1 across the closed intent set.
- **End-to-end system behavior:** pass rate on scenario tests (functional acceptance), out-of-scope (OOS) detection count, and response latency summary (min/median/avg/max).

TABLE III: Intent classification performance (macro-averaged).

Model	Acc.	P	R	F1
Majority baseline	0.146	0.008	0.056	0.014
Keyword rules	0.620	0.646	0.539	0.557
TF–IDF (1-NN cosine)	0.620	0.634	0.539	0.562
TF–IDF + Logistic Regression	0.612	0.652	0.478	0.505
Sentence-Transformers (MiniLM-L6-v2)	**0.783**	**0.773**	**0.771**	**0.756**
Weighted ensemble (0.75 sem + 0.25 TF–IDF)	**0.783**	**0.773**	**0.771**	**0.756**

Macro-averaged scores are emphasized because the intent set is multi-class and exhibits class imbalance; macro scores reduce the risk of majority-class dominance masking poor performance on smaller intents.

B. Experimental Setup

Intent classification evaluation. The evaluation uses the repository intent patterns in `data/intents.json`, which contains approximately 650 preprocessed patterns across 18 intent labels (about 36 patterns per intent on average). A stratified 80/20 split is applied with a fixed random seed (42). The same preprocessing logic used by the implementation (`text_preprocessor.py`) is applied prior to training and evaluation.

Baselines are derived from components available in the repository:

- **Majority baseline:** always predicts the most frequent training intent.
- **Keyword rules:** assigns the intent with the highest keyword match count (using the `keywords` fields in the intent JSON).
- **TF–IDF (1-NN cosine):** nearest-neighbor matching in TF–IDF space.
- **TF–IDF + Logistic Regression:** multi-class linear classifier over TF–IDF uni/bi-grams.
- **Sentence-Transformers semantic similarity:** `all-MiniLM-L6-v2` cosine similarity against training patterns, matching the repository's semantic intent logic.
- **Weighted ensemble:** semantic (0.75) + TF–IDF (0.25) intent selection using the decision rules implemented in `intent_model.py`.

End-to-end system tests. End-to-end results are taken from `TEST_RESULTS_20260407_085552.txt`, which executes 120 scenario tests spanning in-domain requests, hybrid queries, explicit OOS queries, typos, edge cases, and noisy inputs.

C. Performance Results

1) Intent classification (holdout split on repository patterns): **Observations.** Keyword rules improve substantially over the majority baseline, indicating that intent-specific lexical cues exist in the pattern set. Among fast lexical baselines,

TABLE IV: End-to-end automated test outcomes from `TEST_RESULTS_20260407_085552.txt`.

Metric	Value
Total tests	120
Passed	94 (78.3%)
Failed	26 (21.7%)
OOS detected	18
Latency (median / avg / max)	4729.2 / 4520.3 / 14652.3 ms
Functional pass (excluding API failures)	94/106 (88.7%)

TABLE V: 120-scenario suite composition by evaluation bucket.

Bucket	Total	Passed	Pass rate
In-domain task	87	75	86.2%
Robustness/noisy in-domain	22	11	50.0%
Explicit out-of-scope	10	8	80.0%
Boundary inputs	1	0	0.0%

TF–IDF nearest-neighbor provides the strongest macro F1, while logistic regression provides slightly better macro precision. The semantic component provides the strongest performance overall (accuracy 0.783; macro F1 0.756). In this split, the weighted ensemble matches the semantic predictor because the semantic signal dominates under the default weights; this should be interpreted as a calibration/robustness mechanism (confidence smoothing and safer routing under ambiguity), not as an accuracy booster on this holdout. These numbers are best interpreted as pattern-set generalization rather than a full measure of real-user robustness; pattern corpora can contain artifacts that inflate apparent performance [25], [26].

2) End-to-end automated test outcomes (saved execution artifact): The raw 78.3% pass rate includes provider-side batch failures. Excluding 14 API/provider failures yields a functional pass rate of 88.7% on completed responses, which better reflects core system logic. Here, boundary inputs denote deliberately degenerate queries (for example punctuation-only, whitespace-only, or extreme-length gibberish) used to test guardrails rather than standard user behavior. Within the robustness/noisy in-domain bucket, the 11 failures are concentrated in typo-heavy variants (4), vague/underspecified prompts (4), and emotion-heavy short turns (3), explaining the lower 50.0% pass rate.

Taken together, Table III and Table IV show that strong component-level intent metrics do not directly translate to raw batch pass rate when external API reliability is a bottleneck.

D. Latency Breakdown (NLU vs LLM)

The end-to-end artifact reports multi-second response latencies but does not separate local NLU computation from LLM response time. The application implementation exposes an LLM timing signal (`response_time`), enabling a decomposition of per-turn latency into local NLU, LLM inference, and fixed overhead. A local microbenchmark (CPU, after warm-up) indicates that the control layer is comparatively fast: intent-only inference has a median of 27.97 ms (mean 28.55 ms), and intent+emotion+scope has a median of 60.62 ms (mean 63.67 ms; $p95$ 83.86 ms). Consequently, the multi-second tail observed in the end-to-end artifact is primarily attributable to external LLM inference and network variability rather than local intent routing. For a student-facing assistant, median latency around 4.7 seconds is usable but not ideal for conversational flow. Practical mitigation includes token streaming (lower time-to-first-token), cache-first handling for high-frequency intents, and asynchronous UI status updates before full response generation completes.

TABLE VI: Failure attribution for the 26 failed end-to-end tests.

Failure type	Count
API/provider failures (`Both APIs failed`)	14
Boundary/underspecified/noisy inputs	9
Intent/scope misroutes (non-API)	3

TABLE VII: OOS detection metrics on explicit OOS-labeled scenarios.

Metric	Value
Precision	0.444
Recall	0.727
False-positive rate (in-scope flagged OOS)	0.092

E. Statistical Strength of Scenario Tests

The recorded end-to-end scenario result (94/120 passed) corresponds to a raw pass rate of 78.3%. The 95% Wilson confidence interval for this raw operational pass rate is [70.1%, 84.8%]. Because 14/26 failures were API/provider failures, a corrected functional estimate (94/106) is 88.7% with a 95% Wilson interval of [81.2%, 93.4%]. Reporting both values separates infrastructure reliability from core assistant logic.

This expanded 120-scenario evaluation includes systematic coverage of paraphrases, typos, borderline OOS queries, hybrid queries, emotional language, vague inputs, and true out-of-scope boundary cases, reducing measurement artifacts and providing more robust evidence of system behavior in realistic deployment conditions [21], [25]–[27].

F. OOS Detection Metrics

Using explicit OOS-labeled scenarios in the suite (OUTSCOPE + boundary cases), scope-gating outcomes are shown in Table VII.

These values indicate conservative but imperfect OOS routing: recall is acceptable, but precision shows over-rejection on some ambiguous/noisy in-scope queries. Concretely, OOS precision 0.444 corresponds to 8 true OOS catches and 10 false OOS flags; this indicates a recall-leaning operating point and explains why ambiguous/noisy in-domain turns are occasionally over-rejected.

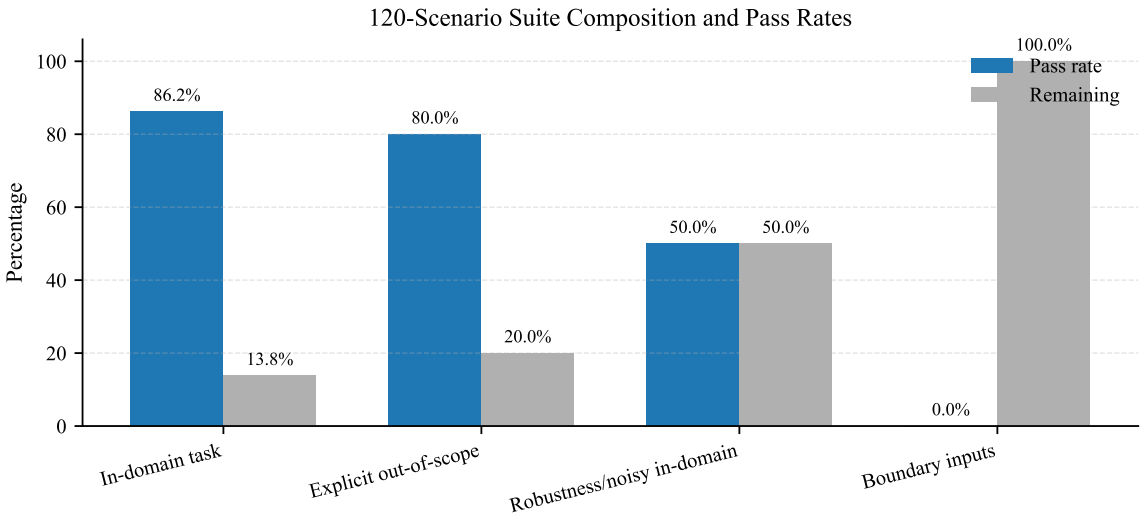

Fig. 2: Grouped pass-rate breakdown across the 120-scenario evaluation buckets.

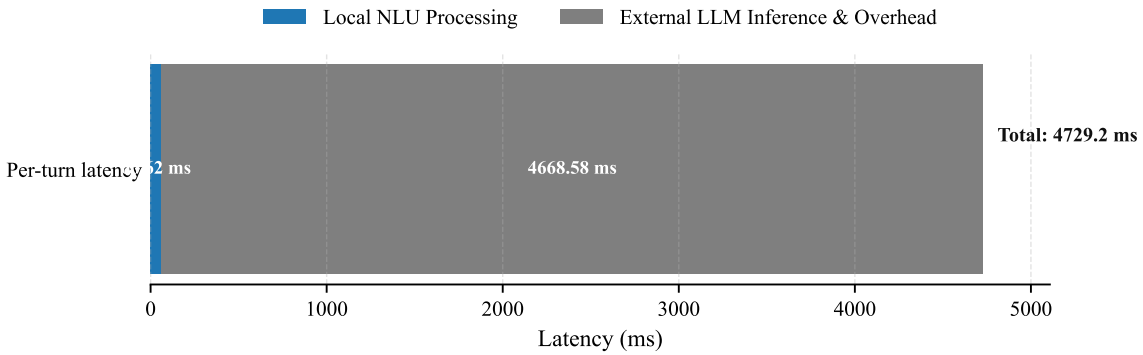

Fig. 3: Per-turn latency breakdown for the median response path. External LLM inference and overhead dominate the total budget.

G. Comparison with Baseline Methods (if available)

The component-level comparison suggests that rule-based routing can be competitive in precision but may be limited in recall, reflecting reduced coverage for paraphrases and short queries. TF–IDF baselines improve macro recall and macro F1, supporting the use of statistical models for intent routing.

The repository implementation additionally includes a semantic similarity component (Sentence-Transformers) and a weighted ensemble. On the same holdout split, the semantic component substantially improves macro F1 (0.756), consistent with prior work emphasizing robust representations for intent routing and uncertainty-aware scoping [19], [21].

Because ensemble and semantic-only scores are identical in this split, the ensemble should be interpreted primarily as an operational reliability design choice (confidence calibration and safer routing under ambiguity) rather than a direct accuracy gain mechanism.

TABLE VIII: Top misclassified intent pairs (TF–IDF + Logistic Regression).

True intent	Predicted intent	Count
admission	eligibility	3
admission	exams	3
gratitude	greetings	2
admission	out_of_scope	2
fees	campus_life	2

H. Error Analysis

Top intent confusions (TF–IDF + Logistic Regression holdout). Representative high-frequency confusion pairs are shown in Table VIII.

End-to-end failure modes. Failure attribution indicates three regimes: API/provider failures (14), boundary/underspecified/noisy queries (9), and non-API intent/scope misroutes (3). This indicates that raw pass-rate degradation is primarily operational during batch execution, while logical failures are

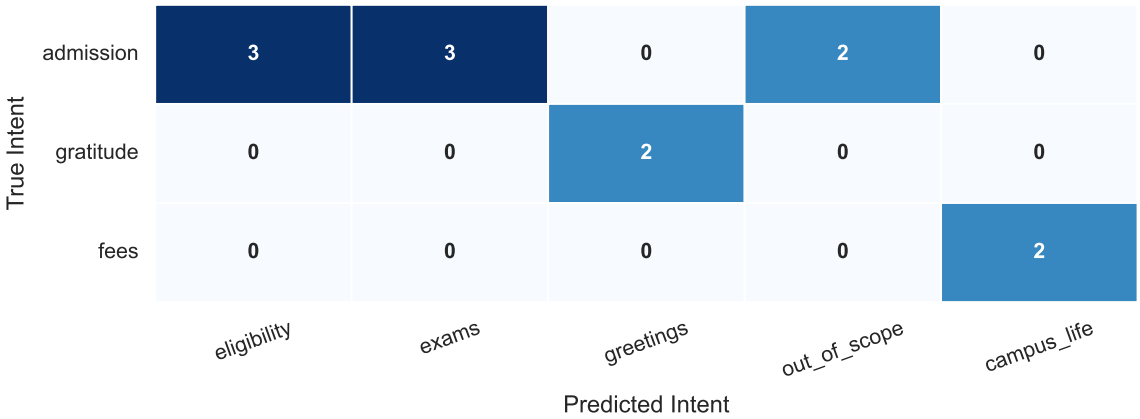

Fig. 4: Confusion heatmap for the top misclassified intent pairs in the TF–IDF + Logistic Regression holdout analysis.

TABLE IX: Bottom-5 per-class F1 (semantic model, holdout split).

Intent	F1	Support
general_info	0.400	4
comparison	0.444	5
affirmation	0.667	5
eligibility	0.667	7
library	0.667	4

concentrated in ambiguity-heavy edge cases [28], [29].

Strengths and weaknesses. Strength is a strong end-to-end functional pass rate on in-domain and OOS scenarios in the recorded artifact, indicating stable orchestration and conservative fallback behavior. Weakness is intent confusion for conversational-act intents and semantically overlapping fee/hostel/admission categories; improving label definitions, increasing pattern coverage, and adding explicit conversational-act handling are expected to improve macro recall.

V. Conclusion

This work addresses the problem of delivering a reliable college-domain assistant that can route user queries to appropriate intents, handle out-of-scope (OOS) inputs conservatively, and produce helpful responses while reducing brittle behavior in real-world interaction.

Approach. The system is implemented as a controlled conversational pipeline that combines preprocessing, ensemble intent inference, scope enforcement, emotion and tone cues, and an LLM response layer with knowledge grounding and operational safeguards (e.g., fallback and recovery). The design reflects evidence that robust deployed assistants require both strong representations and explicit control mechanisms, particularly for OOS handling and noise resilience [19], [21], [28].

Key findings. Quantitative results on repository artifacts indicate: (i) lexical baselines achieve up to 0.620 accuracy and up to 0.562 macro F1 on the held-out split of intent patterns, while the semantic component achieves 0.783 accuracy and 0.756 macro F1 on the same split; and (ii) end-to-end scenario tests on an expanded 120-scenario benchmark achieve a raw pass rate of 78.3% (94/120; 95% CI [70.1%, 84.8%]), with 14 failures attributable to API/provider errors and a corrected functional pass estimate of 88.7% (94/106; 95% CI [81.2%, 93.4%]). In condensed form, the main novelty is methodological: reporting raw operational reliability alongside infrastructure-corrected functional reliability, so evaluation reflects both deployment reality and core assistant logic.

Contributions.

- A repository-grounded multi-signal architecture combining local NLU control and external LLM generation, validated on 120 end-to-end scenarios.
- A deployment-realistic evaluation protocol that reports both raw operational pass rate (78.3%, 94/120) and corrected functional pass rate (88.7%, 94/106).
- A quantified reliability analysis with explicit failure attribution (14 API failures vs 12 non-API failures), OOS metrics (precision 0.444, recall 0.727), and confidence intervals for both reporting modes.

Limitations.

- Intent evaluation is based on locally authored patterns rather than large-scale external benchmarks, and may overestimate generalization due to dataset artifacts [25], [26].
- The pattern inventory is relatively shallow per class (about 36 patterns per intent on average), with low-sample intents (e.g., `unknown`, `campus_life`, `greetings`, `gratitude`, `general_info`) that increase class imbalance effects.
- The intent evaluation reflects pattern-set generalization and does not fully capture real-user distribution shift, multi-turn ambiguity, or adversarial/noisy inputs.
- Boundary input handling (e.g., punctuation-only and whitespace-only) remains a failure point and motivates stronger normalization and routing rules.

Future work.

- Multilingual support: extend token normalization maps in `text_preprocessor.py` for Konkani and Marathi variants observed in the local student population, then re-evaluate OOS false positives on code-mixed queries.
- Better context handling: add anaphora-aware follow-up resolution in `context_manager.py` for pronoun-heavy turns and re-measure failures in the CONTEXT bucket.
- Real-time deployment: implement streaming-first responses in `app.py` and cache-first replies in `llm_handler.py`, then benchmark time-to-first-token and median turn latency.
- Improved robustness: tune τ/γ against a held-out OOS calibration set and re-run the 120-scenario suite to improve OOS precision without collapsing recall.

REFERENCES

[1] S. Hussain, O. A. Sianaki, and N. Ababneh, "A survey on conversational agents/chatbots classification and design techniques," in *Web, Artificial Intelligence and Network Applications: Proceedings of the Workshops of the 33rd International Conference on Advanced Information Networking and Applications (WAINA-2019)*. Springer, 2019, pp. 946–956.

[2] G. Caldarini, S. Jaf, and K. McGarry, "A literature survey of recent advances in chatbots," *Information*, vol. 13, no. 1, p. 41, 2022.

[3] J. Gao, M. Galley, and L. Li, "Neural approaches to conversational ai," *Foundations and Trends in Information Retrieval*, 2019.

[4] B. Liu and I. Lane, "Joint intent classification and slot filling with recurrent neural networks," in *Proceedings of Interspeech*, 2016.

[5] D. Hakkani-Tur *et al.*, "Multi-domain joint semantic frame parsing using bi-directional rnn-lstm," in *Proceedings of Interspeech*, 2016.

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017.

[7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019.

[8] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[9] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," OpenAI, Tech. Rep., 2018.

[10] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, Tech. Rep., 2019.

[11] T. B. Brown *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, 2020.

[12] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems*, 2022.

[13] R. Thoppilan *et al.*, "Lamda: Language models for dialog applications," *arXiv preprint arXiv:2201.08239*, 2022.

[14] OpenAI, "Chatgpt: Optimizing language models for dialogue," https://openai.com/blog/chatgpt, 2022.

[15] C. Khatri *et al.*, "Advancing the state of the art in open domain dialog systems through the alexa prize," in *Proceedings of Alexa Prize Workshop*, 2018.

[16] S. Roller *et al.*, "Recipes for building an open-domain chatbot," in *Proceedings of ACL*, 2021.

[17] Q. Chen, Z. Zhuo, and W. Wang, "Bert for joint intent classification and slot filling," *arXiv preprint arXiv:1902.10909*, 2019.

[18] C. Xia, C. Zhang, X. Yan, Y. Chang, and P. Yu, "Capsule neural networks for joint slot filling and intent detection," in *Proceedings of ACL*, 2018.

[19] I. Casanueva, T. Temcinas, D. Gerz, M. Henderson, and I. Vulic, "Efficient intent detection with dual sentence encoders," in *Proceedings of the 2nd Workshop on NLP for Conversational AI*, 2020, pp. 38–45.

[20] C. Zhang *et al.*, "Few-shot intent detection via contrastive pretraining and fine-tuning," in *Proceedings of EMNLP*, 2020.

[21] S. Larson, A. Mahendran, J. Peper, C. Clarke, A. Lee, P. Hill, J. K. Kummerfeld, K. Leach, M. Laurenzano, L. Tang, and J. Mars, "An evaluation dataset for intent classification and out-of-scope prediction," in *Proceedings of EMNLP-IJCNLP*, 2019, pp. 1311–1316.

[22] S. Kim *et al.*, "Dialogue state tracking: A neural reading comprehension approach," in *Proceedings of SIGDial*, 2019.

[23] C.-W. Huang *et al.*, "Transferable multi-domain state generator for task-oriented dialogue systems," in *Proceedings of ACL*, 2020.

[24] W. Maroengsit, T. Piyakulpinyo, K. Phonyiam *et al.*, "A survey on evaluation methods for chatbots," in *Proceedings of the 2019 7th International Conference on Information and Education Technology*. ACM, 2019.

[25] S. Gururangan, S. Swayamdipta, O. Levy, R. Schwartz, S. R. Bowman, and N. A. Smith, "Annotation artifacts in natural language inference data," in *Proceedings of NAACL-HLT*, 2018, pp. 107–112.

[26] M. Geva, Y. Goldberg, and J. Berant, "Are we modeling the task or the annotator? an investigation of annotator bias in natural language understanding datasets," in *Proceedings of EMNLP-IJCNLP*, 2019, pp. 1161–1166.

[27] S. Swayamdipta, R. Schwartz, N. Lourie, Y. Wang, H. Hajishirzi, N. A. Smith, and Y. Choi, "Dataset cartography: Mapping and diagnosing datasets with training dynamics," in *Proceedings of EMNLP*, 2020, pp. 9275–9293.

[28] Y. Belinkov and Y. Bisk, "Synthetic and natural noise both break neural machine translation," in *Proceedings of ICLR*, 2018.

[29] Z. Sun, A. Li, and X. Qiu, "Adversarial training for code-mixed language understanding," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 1364–1373.

[30] J. Gao, M. Galley, and L. Li, "Recent advances in conversational ai," *AI Magazine*, 2020.

[31] T.-H. Wen *et al.*, "A network-based end-to-end trainable task-oriented dialogue system," in *Proceedings of EACL*, 2017.

[32] A. Bordes, Y.-L. Boureau, and J. Weston, "Learning end-to-end goal-oriented dialog," in *Proceedings of ICLR*, 2017.

APPENDIX

A. Reproducibility Checklist

Item	Value
OS	Windows
Python	3.10+ (CPython)
Main dependencies	streamlit 1.28.1, torch 2.1.1, transformers 4.35.2, sentence-transformers 2.2.2, scikit-learn 1.3.2, pandas 2.1.4, numpy 1.24.3
API dependencies	groq 0.13.3, google-genai 0.1.0, requests 2.31.0
Testing/tooling	pytest 7.4.3, pytest-cov 4.1.0, pytest-xdist 3.5.0, pytest-timeout 2.1.0, hypothesis 6.92.0
Dataset files	`data/intents.json`, `data/college_data.json`
Primary result artifact	`TEST_RESULTS_20260407_085552.txt`
Intent split protocol	Stratified 80/20, random seed 42

TABLE X: Minimal reproducibility checklist for rerunning evaluation and figures.

B. 120-Scenario Inventory (Machine-Readable Artifacts)

The complete scenario inventory and execution traces are provided in: `CHATBOT_QUESTION_TEST_CASES.txt`, `STREAMLIT_TEST_CASES.txt`, and `TEST_RESULTS_20260407_085552.txt`. Bucket counts used in this report are: in-domain task (87), robustness/noisy in-domain (22), explicit OOS (10), and boundary inputs (1).

C. OOS Confusion Details and Formulae

Using explicit OOS-labeled scenarios: $TP = 8$, $FP = 10$, $FN = 3$, $TN = 99$.

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{8}{8 + 10} = 0.444$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{8}{8 + 3} = 0.727$$

$$\text{FPR} = \frac{FP}{FP + TN} = \frac{10}{10 + 99} = 0.092$$

D. Figure Generation Provenance

Figure scripts and outputs:

- `Paper/generate_latency_breakdown_chart.py` → `per_turn_latency_breakdown.(pdf|svg)`
- `Paper/generate_confusion_matrix_heatmap.py` → `top_misclassified_intents_heatmap.(pdf|svg)`
- `Paper/generate_suite_composition_passrates_char` → `suite_composition_pass_rates.(pdf|svg)`

Regeneration commands:

```
python Paper/generate_latency_breakdown_chart.py
python Paper/generate_confusion_matrix_heatmap.py
python Paper/generate_suite_composition_passrates_
```

E. Representative Failure Log Excerpts

Query (excerpt)	Test Type	Detected Intent	Failure Regime
Wht r the fess?	`fees_-_with_typos`	hostel (77.9%)	Noise/typo misrouting
Can I retake the exam if I fail?	`exam_-_re-exam_opportunity`	exams (100.0%)	Retrieval/generation mismatch
What's the average salary for placements?	`placements_-_average_package`	placements (95.1%)	Knowledge grounding gap
Do placements vary by specialization/branch?	`placements_-_specialization_gaps`	placements (71.1%)	Specificity gap
What if I want to pursue higher studies instead?	`placements_-_higher_studies`	comparison (44.0%)	Intent overlap
How many students can be accommodated in hostel?	`hostel_-_capacity`	hostel (87.4%)	Missing structured field
What labs are available for students?	`facilities_-_labs`	fees (46.0%)	Cross-intent confusion
How do I apply to the college?	`admission_-_application_process`	admission (65.6%)	Incomplete response
What types of scholarships are available?	`scholarship_-_types_available`	fees (72.1%)	Intent schema overlap
Wat about the exams skeduled?	`typo_-_phonetic_spelling`	exams (100.0%)	Noise robustness failure

TABLE XI: Representative failed cases from the latest 120-scenario execution artifact.

F. Prompt/Policy Snapshot (Compact)

Runtime control policies used in deployment evaluation: (i) scope-first routing before generation, (ii) confidence-aware clarification via dynamic thresholds, (iii) controlled prompt construction with intent/scope/emotion/grounding fields, and (iv) reliability safeguards combining cache lookup, throttling, concurrency limits, jittered backoff, API-key rotation, and Groq-to-Gemini fallback.